

RESEARCH

Open Access



Comparative analysis of AI-based search algorithms in solving 8 puzzle problems

Ghaniyyat Bolanle Balogun¹, Daniel Ibisagba¹, Amos Bajeh¹, Taofik Olawale Debo¹,
Abdulraheem Muyideen¹ and Olumuyiwa James Peter^{2*} 

Abstract

Background The 8-puzzle problem is a well-known combinatorial search problem, often used to test the effectiveness of various artificial intelligence (AI) algorithms. This study aims to evaluate the performance of three AI-based search algorithms—Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search—in solving the 8-puzzle problem. The algorithms were implemented in Java, and their performance was measured in terms of solution length, the number of expanded nodes, and execution time.

Results Experiments were conducted on eight randomly generated instances of the 8-puzzle problem. The findings reveal that both BFS and A* Search consistently identify optimal solutions in all cases. However, BFS was found to use more memory and operate slower than A* Search. Conversely, DFS demonstrated faster execution times and lower memory usage compared to BFS but was less reliable in finding optimal or feasible solutions. The study also explored the impact of different heuristic functions on the performance of A* Search. Among the heuristics tested, the Manhattan distance heuristic was determined to be the most effective, offering the best balance between accuracy and efficiency.

Conclusions In conclusion, both BFS and A* Search are effective in finding optimal solutions, with A* Search being more efficient in terms of speed and memory usage. DFS, while faster and more memory-efficient, is less consistent in producing optimal solutions. The Manhattan distance heuristic significantly enhances the performance of A* Search, making it the preferred heuristic for the 8-puzzle problem. These results suggest that A* Search, particularly with the Manhattan distance heuristic, is a highly efficient choice for solving the 8-puzzle problem, especially in contexts where computational resources are limited.

Keywords Artificial intelligence, Based search algorithms, Heuristics function

Background

The application of Artificial Intelligence (AI) to problem-solving has transformed several industries. AI can be characterized in many ways, including trying to replicate human intelligence or solving problems at the human

level using any approach, even if it differs greatly from human problem-solving strategies. Writing programs to produce something “novelty” in the sense of coming up with a fresh approach, method, or technique to solve a problem that a human programmer could not have imagined is another way to characterize artificial intelligence (Gulak 2024). The programmer needs to take action to initiate this process of discovery. A common method to achieve “novelty” in an AI challenge is to frame it as a search problem, creating a “space” for all potential solutions and then creating an algorithm to search that space for a good solution. If the “space” is sufficiently tiny, we can search it thoroughly until we are certain that we have

*Correspondence:

Olumuyiwa James Peter
peterjames4real@gmail.com

¹ Department of Computer Science, Faculty of Communication and Information Sciences, University of Ilorin, Ilorin, Nigeria

² Department of Mathematical and Computer Sciences, University of Medical Sciences, Ondo City, Ondo State, Nigeria

found the ideal answer. Search algorithms are one of the main uses of AI. AI-based search algorithms traverse a search space in an attempt to identify the best solution to a problem (Omonkhodion 2023; Pathak et al. 2018).

State Space: This is a process used in computer science to identify a goal and the path to the target state that has the necessary configuration. It is particularly useful in the field of artificial intelligence when progressive designs or states of a given state are taken into consideration. A state space, or collection of states into which the problem can be arranged, is frequently used as a model for search problems. If there is an operation

that can change the current state into the other state, the set of states with connections between the two states forms a tree (Nayak 2014).

A search tree in the 8 puzzle problem is a tree data structure that depicts every potential state of the puzzle, (Mishra and Siddalingaswamy 2017). The puzzle's starting state is represented by the tree's root; while, the desired states are represented by the tree's leaves. The puzzle's intermediate states are represented by the interior nodes of the tree. Figure 1 shows a typical search tree of the 8-puzzle problem; while, Table 1 shows algorithms comparison table.

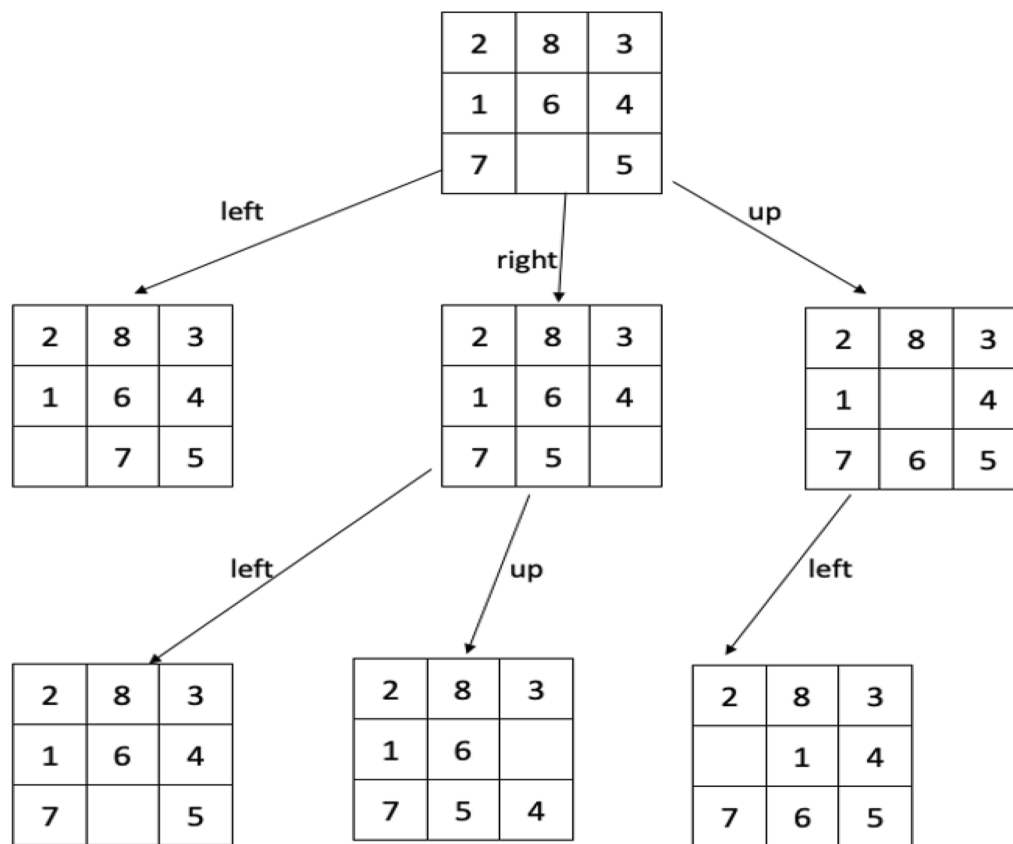


Fig. 1 A typical search tree of the 8-puzzle problem

Table 1 Algorithms comparison table

Approach	Time complexity	Space complexity	Completeness	Optimality	Admissibility
Breadth-first search (BFS)	$O(B^D)$	$O(B^D)$	Yes	Yes	Yes
Depth-first search (DFS)	$O(B^M)$	$O(BM)$	No	No	–
A* search with misplaced tile heuristic	$O(B^D)$	$O(B^D)$	Yes	Yes	Yes
A* search with Manhattan distance heuristic	$O(B^D)$	$O(B^D)$	Yes	Yes	Yes

The 8 puzzle problem

Noyes Palmer Chapman created and popularized the 8-puzzle problem, a kind of sliding block puzzle, in 1870. It is made up of a blank square and eight square tiles, numbered 1 through 8, arranged in a 3-by-3 grid. It is a condensed form of the 15-puzzle challenge.

The 8 puzzle issue can be represented as a state space search problem, in which every action is the movement of the tile next to the blank square into the blank square, and every state is a configuration of the tiles on the grid. A starting or initial position and a predetermined goal or target position make up the game. The objective is to rearrange the tiles so that the starting position becomes the desired position. Any random arrangement of the tiles is the initial state; an ordered configuration of the tiles is the goal state. Every action is considered to have a single unit of cost.

The 8-puzzle issue is a search problem, with the desired or target configuration represented by the goal state and the starting configuration of the tiles represented by the start state. Applying a series of allowable moves to advance from the initial or start state to the objective state can solve the problem. One of the authorized moves is to slide a tile into the space next to it. The objective is to determine the shortest path that gets to the desired state.

The eight-puzzle problems can be resolved using a variety of search techniques, including depth-first search (DFS), breadth-first search (BFS), and A* search. Regarding optimality, completeness, space complexity, and time complexity, each algorithm has pros and cons. While some algorithms may fail to explore the full state space or become caught in loops, others are certain to discover a solution if one exists. While some algorithms might find less-than-ideal answers or none at all, others promise to find the best solution in the fewest moves possible (Meister 2020; Navya and Ranjith 2021). Figure 2 show A pictorial representation of the 8-puzzle problems.

Algorithms for artificial intelligence search

By navigating a search space that contains every state conceivable for the problem, search algorithms are utilized to find solutions for a variety of issues. Uninformed search algorithms and informed search algorithms are

the two basic categories to which these algorithms often belong.

Blind search algorithms, also known as ignorant search algorithms, function by not employing any extra problem-specific data to guide the search procedure. Rather, they scan the search area methodically to find a solution. Among the uninformed search algorithms' subcategories are:

1. Breadth-first search
2. Depth-first search
3. Uniform cost search

Breadth-first search: This type of search method is based on the breadth first traversal that may be found in graphs and trees. With this method, the initial state of the issue is treated as the root, and the search space is shown as a collection of nodes that arise from a single move or operation on a particular node to its nearby neighbors. The Breadth-First Search algorithm, which follows the "First In First Out" tenet, makes use of the queue data structure. Depending on the issue domain and the designated goal state, the time complexity can be estimated linearly with the number of vertices and edges. There can be any number of edges, ranging from one to the square of the number of vertices (Awerbuch 1985).

The depth first traversal found in trees and graphs provides the foundation for the depth-first search technique. This technique treats the starting state of the problem as the root and represents the search space as a set of nodes that result from a single move or operation on a given node to its immediate neighbors. When the ideal state, or goal state, is located far down a branch, this algorithm works well; unlike Breadth-First Search, it does not require going through every level to get there (Tarjan 1972). The stack data structure used by the Depth First Search algorithm adheres to the "Last In Last Out" principle (Zhang et al. 2021).

The Uniform Cost Search (UCS) technique is designed to discover the least expensive path between a start node and a goal node in a weighted graph. It investigates nodes with the lowest total cost and ranks the lowest-cost nodes according to a priority queue. Though it can be computationally costly in large graphs, UCS ensures optimality (Norvig and Russell 2010).

Heuristic search algorithms, sometimes referred to as informed search algorithms, direct the search for a solution using extra problem-specific knowledge, such as a heuristic function. The Heuristic function in this instance, represented by $h(n)$, calculates the separation between the current state and each potential state and the intended goal state. The following categories can be used to separate these algorithms:

Initial State			Goal State		
1	2	3	2	8	1
8		4		4	3
7	6	5	7	6	5

Fig. 2 A pictorial representation of the 8 puzzle problem

1. Greedy Search.
2. A* Search.
3. Graph Search.

Greedy Search: Greedy search is a search method that finds the most plausible path toward a goal node based on a heuristic function. It is utilized in graph theory and AI-based problem-solving (Norvig and Russell 2010). It ignores the cost and distance already traveled in favor of growing nodes that seem to be closest to the desired state. Greedy Search makes the most optimistic decision by concentrating only on the heuristic evaluation at each stage. Nevertheless, this method may become trapped in local optima and does not ensure the discovery of the best solution.

A* Search: Because of its improved search performance and accuracy in locating the specified answer inside the search space, the A* (pronounced A-star) algorithm is a well-liked option for resolving search issues. Most people agree that this algorithm performs noticeably better than other search methods. Usually, other algorithms that can preprocess the search space outperform the algorithms that have been demonstrated to outperform the A-star technique because they have the advantage of knowing exactly which path to take to reach the objective state. A-star method must continue in a manner that minimizes the cumulative function of the expenses incurred thus far along the path as well as the heuristic used to determine the cost of the least expensive path from the current node to the goal state. (Nosrati et al. 2012).

The formula for calculating cost is $f(n) = c(n) + h(n)$, where $c(n)$ is the initial cost and $h(n)$ is the heuristic that yields the cost of the least expensive path to the optimal goal state.

Graph search: Graph search is a search algorithm used in graph theory and artificial intelligence that finds a path from a start node to a goal node by methodically examining a network. To prevent redundancy, it keeps track of the nodes that have been examined and manages the frontier of unexplored nodes using a queue or stack. Graph Search can handle graphs with cycles or repeating states and guarantees that each node is accessed only once. On the other hand, if the graph has loops without a target state, it can experience infinite loops (Norvig and Russell 2010).

Heuristics function

To steer the search process by providing an estimate of the desirability or cost from a current state to a goal state, Norvig and Russell 2010 advocated the use of heuristic functions in search algorithms. Prioritizing the investigation of states that have a higher chance of leading to the objective helps. Heuristic functions are employed in

sophisticated search algorithms like A* Search and are usually based on domain knowledge or problem-specific knowledge. A heuristic's accuracy in estimating the cost or distance to the desired state is what defines its quality. In this study, two primary heuristic functions are being examined, namely:

Manhattan Heuristic: Often referred to as Taxicab, the Manhattan Heuristic is a well-known and formally described function. It operates on the premise that there are a set number of "moves" (streets, tiles, etc.) that must be made to go from one state to another. As a result, information about how the puzzle should be solved and how tiles "move" between States is provided. The sum of all these transfers from one State to the next is known as the Manhattan Distance Heuristic (Nayak 2014). In general, it is preferable if the next move requires fewer moves (i.e., less money). The Manhattan Distance can be expressed as $h(n)$, where n is the total number of moves required to place every tile in the Goal State.

Euclidean Distance Heuristic: This heuristic uses the geometry-based Euclidean distance formula to determine the straight-line distance between the present state and the desired state. It calculates the shortest path in a plane between two points. This heuristic gives a clear estimate of the distance to the desired state and is frequently used in situations involving unconstrained mobility.

Review of related literature

The most difficult puzzle of its type that can be solved completely is the 8-puzzle. It has a large combinatorial problem space of $9!/2$ states despite its simplicity. The size of the search space presents the biggest obstacle to addressing the problem because of its complexity, necessitating the employment of a heuristic search strategy (Shi 2011).

Numerous individuals have attempted to identify the top-performing algorithms utilized in resolving numerous AI-related issues. An eight-puzzle problem is a nice illustration. Based on their time and space complexity and completeness, Pathak et al. (2018), compare the following search strategies: breadth-first search, depth-first search, uniform cost search, A*, and greedy search. Several search methods have been compared by Iordan (2016) and Menon and Amali (2018), in their solutions to the 8-puzzle challenge. Menon and Amali (2018), concentrated on the quantity of nodes investigated and the amount of time needed to solve the puzzle; whereas, Iordan (2016), examined the effective branching factor and running time. It was discovered that applying the Manhattan heuristic, which is more advanced than the Hamming heuristic, improved the spatial complexity of the A* algorithm in comparison with the Hamming heuristic (Iordan 2018). In two instances, the fewest

number of movements necessary to achieve optimal solutions were achieved. The Manhattan heuristic is said to be more informed than the Hamming heuristic, which makes it a more effective and streamlined approach for the A* heuristic search, according to both theoretical and experimental evidence.

It is known that the $N \times N$ form of the 8-puzzle is NP-hard, meaning that it can take exponentially long to find the best solution used the 8-puzzle as a benchmark model in their follow-up experiments to evaluate the benefits of node ordering systems in Iterative-Deepening A* (IDA*). A key discovery of their research is that the majority of IDA* implementations performed worse than what was possible with a simple random operator ordering technique. To make sure the best answer is found, the A* algorithm is applied. To allow the algorithm to find the best solution by examining the fewest possible states, various heuristics are examined (Kunkle, n.d.).

According to Chowdhary and Chowdhary (2020), algorithm B is deemed optimal if no other search algorithm can complete the task in less time or space, or by extending fewer nodes while preserving the same level of quality of solution as algorithm A.

However, a search algorithm's performance is highly dependent on the problem it is trying to solve. For a particular instance, different algorithms work differently. Consequently, it is essential to research how search algorithm performance compares across different case studies.

Game developers can select the best search algorithm to tackle 8-puzzle problems and other related search and path finding challenges in the field of computer game development by determining which search algorithms are the most efficient. This will significantly reduce the amount of memory and processing power needed for producing games. It is particularly useful for low-resource devices.

Methods

The different stages involved in implementing the methodology of this research include the;

- algorithms implementation and
- criteria for evaluation of each algorithm

Testing approach

Testing was done on the PuzzleState as the first phase in the application development process. Tests were prepared before methods were implemented in the classroom using JUnit. This strategy guaranteed each method's accuracy and flexibility in response to modifications while preserving positive test results. Cross-testing

was used in the testing procedure, in which several tests were created to interact with one another in addition to evaluating the PuzzleState. The comprehensive testing methodology was expanded to encompass other classes that deal with data manipulation, ensuring the accuracy of the methods as standalone components and as part of the overall program. Black box testing was carried out by running the application with various inputs and viewing the subsequent outputs after it was determined to be functionally correct. The number of enlarged states, the duration of each method, and the best Search strategy in terms of speed and accuracy were among the important metrics assessed. Figure 3 shows the Methodology Framework; while, Figure Demonstrate test instances on the space tree.

Assessment criteria

The following assessment measures were employed in this study:

- *Completeness* Does the algorithm always find a solution?
- *Optimality* Does the method find the problem's shortest or most effective solution?
- *Time complexity* What is the duration required by the method to discover a solution?
- *Space complexity* What is the algorithm's memory usage?

The relative relevance of these characteristics will change depending on the specific application. For instance, even though an algorithm takes longer to run, there are situations when it may be more crucial for it to be complete and optimal. In some situations,

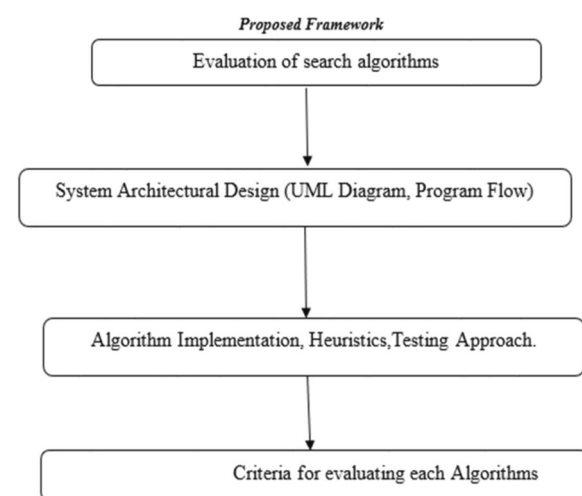


Fig. 3 Methodology framework

an algorithm's speed and usability may be more crucial than its completeness or optimality.

The 8 puzzle problem's main difficulty is its intricacy. Even with its tiny grid size, the issue is part of a broad class of sliding block puzzles whose complexity grows exponentially with increasing grid size. The number of viable movements and states increases quickly, making it challenging to identify the best course of action. There is a significant branching factor because each block can go in up to four different directions: left, right, down, and up.

Review of algorithms

The initial step in the search procedure for each of the implemented algorithms is to visit the root node of the search tree that is supplied by the starting state. To visit a node, three main tasks must be completed in order:

- taking the node out of the boundary set.
- assess whether a solution has been found by comparing the current state to the desired state.

Ultimately, if the check returns a negative result, the node is enlarged. Figure 4 Demonstrate test instance on space tree.

By creating successor nodes close to it and including them in the frontier set, one can enlarge a given node. Remember that once these successor nodes are present or have been visited, they should not be added to the frontier once more. The figure demonstrates test instance on space tree.

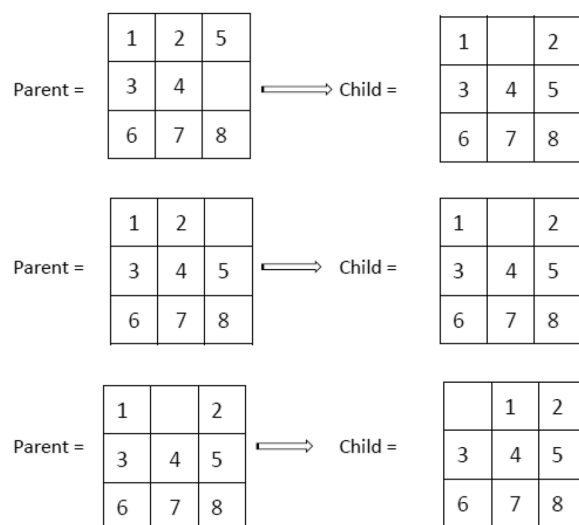


Fig. 4 Demonstrating test instance on space tree

Algorithm test instances

Suppose the program is executed for breadth-first search starting from the initial state 1,2,5,3,4,0,6,7,8 as follows.

The program output file contains the following lines:

```
Used search algorithms: BFS
Found goal state: true
Path to Goal: [Up, Left, Left]
Cost of path: 3
Expanded nodes: 10
Depth of search: 3
Maximum depth of search: 4
Running time: 0.0018096010000000003
```

Program illustration using search tree

The 8-puzzle was solved using various algorithms, beginning with different initial states, as seen in the following figures and animations. For A* the Manhattan heuristic is been used, which is an admissible heuristic for this problem. The search paths from the starting state to the goal node are also shown in the figures (the states with red text indicate the selected path). First, let's look at a really basic illustration. Clearly, in this straightforward example, every algorithm determines the same route from the starting state to the goal node.

Example 1: initial state—1,2,5,3,4,0,6,7,8

BFS: The path to the goal node with BFS is shown in Fig. 5.

Figure 5 displays the nodes that BFS expanded, as well as the nodes that are at the edge or frontier of the queue.

Figure 6 shows full expanded nodes search Tree Fig. 7 depicts the path to the goal node with DFS; while, Fig. 8 shows A* search Tree. Also, Fig. 9 shows A* search Tree for second problem Instance.

A*: Figure 8 illustrates the path to the destination node with A* (as well as the nodes expanded by A*).

Example 2: Initial state—1,4,2,6,5,8,7,3,0

A*: The path to the goal node with A* is shown in Fig. 9.

Figure 10 displays every node that A* expanded, even the nodes that are at the edge or frontier of the queue; while, Fig. 11 shows BFS search Tree for second problem instance.

Results

To test the application as a whole in solving a given puzzle problem, the second test case illustrated above, in Example 2, is considered. The aim is to test the

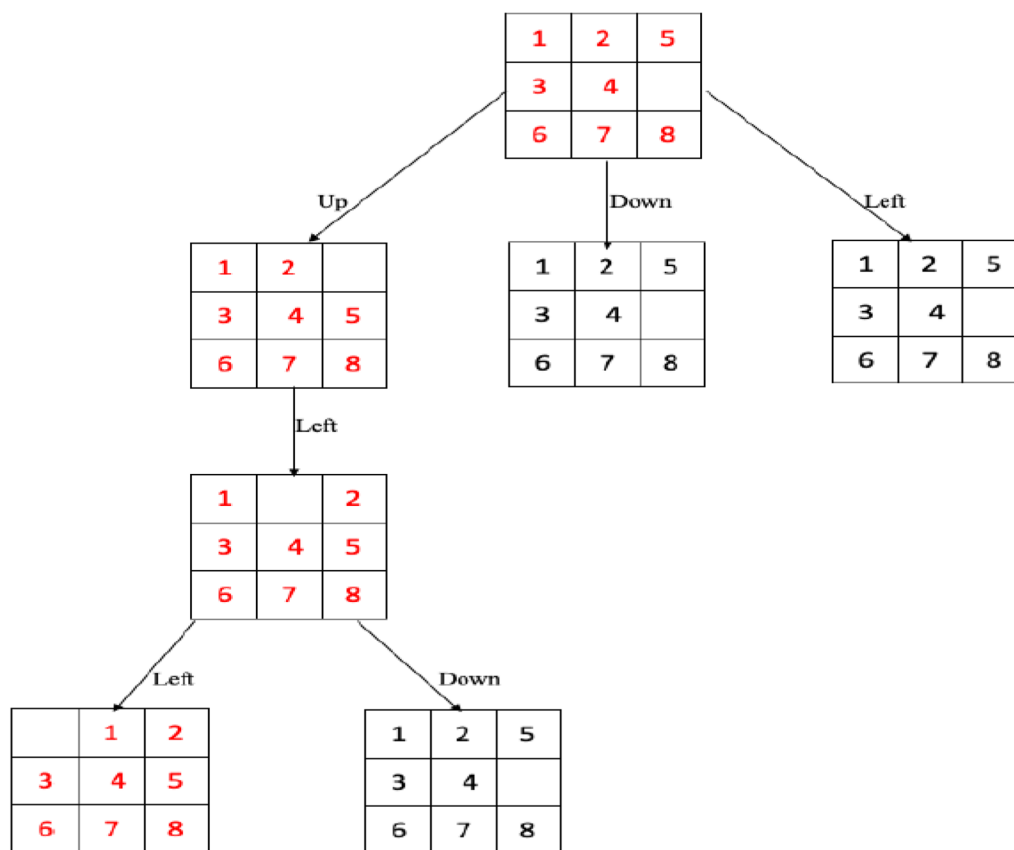


Fig. 5 BFS search tree

theory of each type of search algorithm, its complexities, and the admissibility of the heuristics being implemented. To demonstrate this here, the text file Result of each of the algorithms implemented is displayed. Figures 12, 13 and 14.

Discussion

A comparison chart of various methods for solving the AI 8-puzzle issue is displayed in chart 1. The results show that while DFS requires many more expansions and takes a lot longer to find the solution, BFS finds it relatively quickly (187 expansions). This suggests that DFS is not a good technique in this case because the solution is not too far through “neighboring” states and is not too deep into the state space.

A* appears to locate a solution more efficiently and with fewer steps involved. The Manhattan Distance (14 expansions) outperforms Hamming/Euclidean (16 expansions) when heuristics are compared. As previously mentioned, rather than relying solely on how the grid “looks” with Hamming, the Manhattan distances provide a very accurate estimate of how near a state is to be proper by computing the number of “moves” that must

be made. However, both perform admirably, are comprehensive, and have a very quick runtime, demonstrating their admissibility, completeness, and optimization.

In the well-known AI challenge known as the “8-puzzle problem”, numbered tiles are slid on a 3×3 grid to reach a goal state. This problem can be solved in several ways, such as heuristic search algorithms (A*) and uninformed search algorithms (BFS, DFS). The temporal complexity of the Breadth-First Search algorithm is $O(b^d)$, where b is the branching factor of the search tree (the number of moves that can be done from each state) and d is the destination state’s depth. The 8-puzzle problem has a maximum branching factor of 4 (each tile can be moved in 4 directions) and a maximum depth of 31 (the number of moves required to get from the original state to the target state).

As a result, the algorithm’s time complexity is $O(4^{31})$, an astronomically high figure. Since it is highly probable to identify the objective state before reaching the maximum depth, the algorithm will not really explore the full tree. On the other hand, the algorithm’s running time upper bound is provided by the worst-case time complexity. The size of the search space and the effectiveness

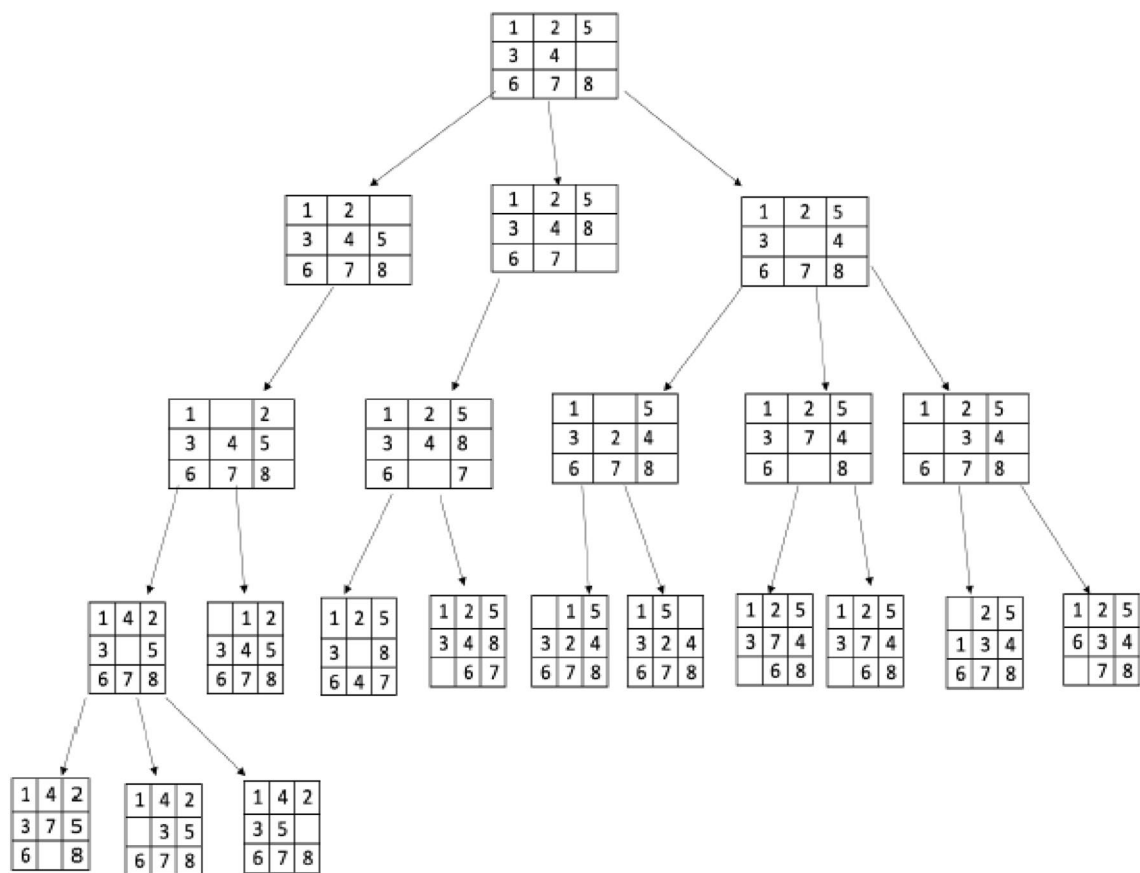


Fig. 6 BFS Full expanded nodes search tree

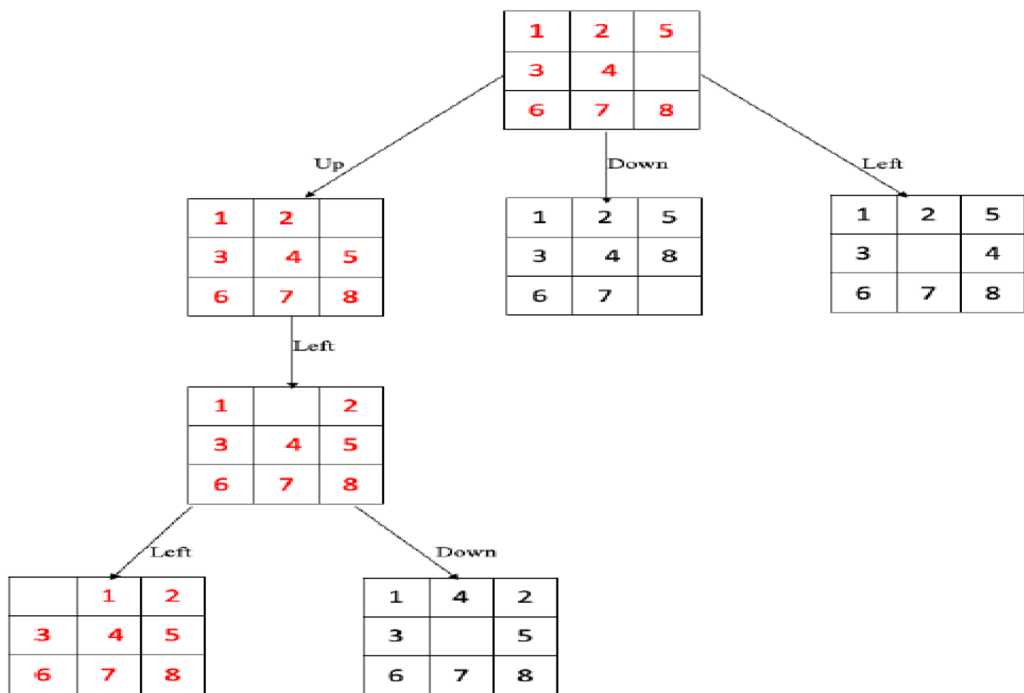


Fig. 7 DFS search tree

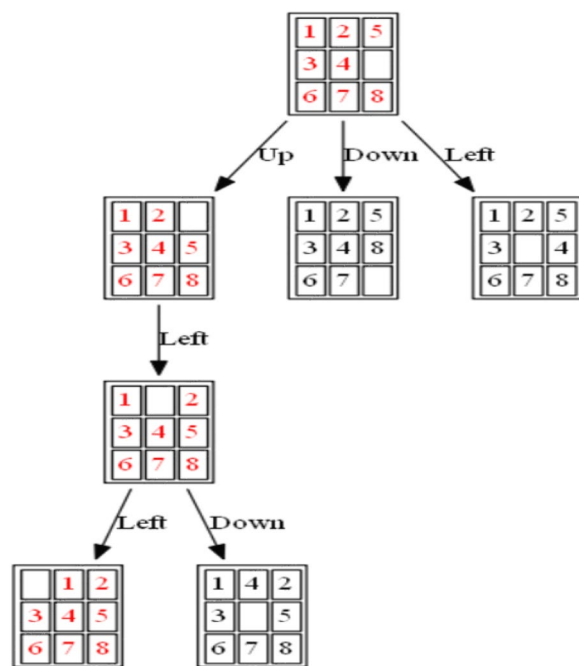


Fig. 8 A* search tree

of the implementation affect the algorithm's execution time. Large or complicated tasks can generally cause the algorithm to run slowly.

On the other hand, the search area for the 8-puzzle issue is really limited, and the efficient method developed can solve the problem on current machines in a fair amount of time. Depending on the starting state and the desired state, the algorithm's execution time may change. In the best scenario, the algorithm will finish rapidly and the goal state can be attained in a few moves. In the worst situation, the algorithm may take a while to identify the desired state due to the size of the search space. Nonetheless, the method ought to be able to locate the answer in an acceptable period for the majority of random initial states and destination states.

The Depth-First Search (DFS) algorithm has an $O(b^m)$ time complexity, where m is the maximum depth of the search tree and b is the branching factor of the search tree (the number of moves that can be made from any state). The 8-puzzle issue has a branching factor of no more than 4 (each tile can be moved in 4 directions), and there is no clear limit depth for the search tree since cycles in the graph allow the search to go on forever. In actuality, though, the search usually ends after a limited number of steps—as long as we remember the states we've visited so we don't end up back there.

The size of the search space and the effectiveness of the implementation affect how long the DFS algorithm takes to execute. In general, since DFS thoroughly examines

the tree before going back, it may occasionally be faster than BFS, particularly if the destination state is located further down the search tree.

DFS may, however, occasionally operate very slowly, particularly in situations when the search space is very wide or the graph has a large number of cycles.

The DFS algorithm's execution time for the 8-puzzle problem varies based on the starting and end states. In the best scenario, the algorithm will finish in a few steps and the desired state can be attained rapidly. In the worst situation, the algorithm may take a while to reach the desired state because the search space may be quite big or contain numerous cycles. Nonetheless, the method ought to be able to locate the answer in an acceptable period for the majority of random initial states and destination states.

By reducing the search space and avoiding returning the same states, optimization techniques like bidirectional search and iterative deepening can enhance the time complexity and execution time of the DFS algorithm.

The structure of the search space and the effectiveness of the heuristic function being utilized determine how complex the A* Search is in terms of time. A* Search will always locate the optimal solution, or the solution requiring the fewest moves, in a finite period if the heuristic function is admissible, which means that it never overestimates the distance to the goal state.

The time complexity of A* Search for the 8-8 puzzle problem is $O(b^d)$, where b is the search tree's branching factor (the number of moves that may be made from each state) and d is the depth of the best answer. Nonetheless, the search space can be reduced by employing an acceptable heuristic function, like the Manhattan distance, which lowers the effective branching factor and, hence, the time complexity. Because A* Search can avoid pursuing numerous unproductive pathways, it can perform considerably quicker than BFS or DFS for huge search spaces.

The size of the search space, the effectiveness of the heuristic function, and the efficiency of the implementation all affect how long the A* Search algorithm takes to execute. Generally speaking, A* Search can execute more slowly than BFS or DFS because it must compute the heuristic function for each node in the priority queue, which can be costly for complex heuristic functions. However, by utilizing an effective data structure, like a binary heap or a Fibonacci heap, the priority queue operations can be completed in logarithmic time, lowering the overall execution time.

The initial state, the goal state, and the effectiveness of the heuristic function employed can all affect how long the A* Search algorithm takes to solve the

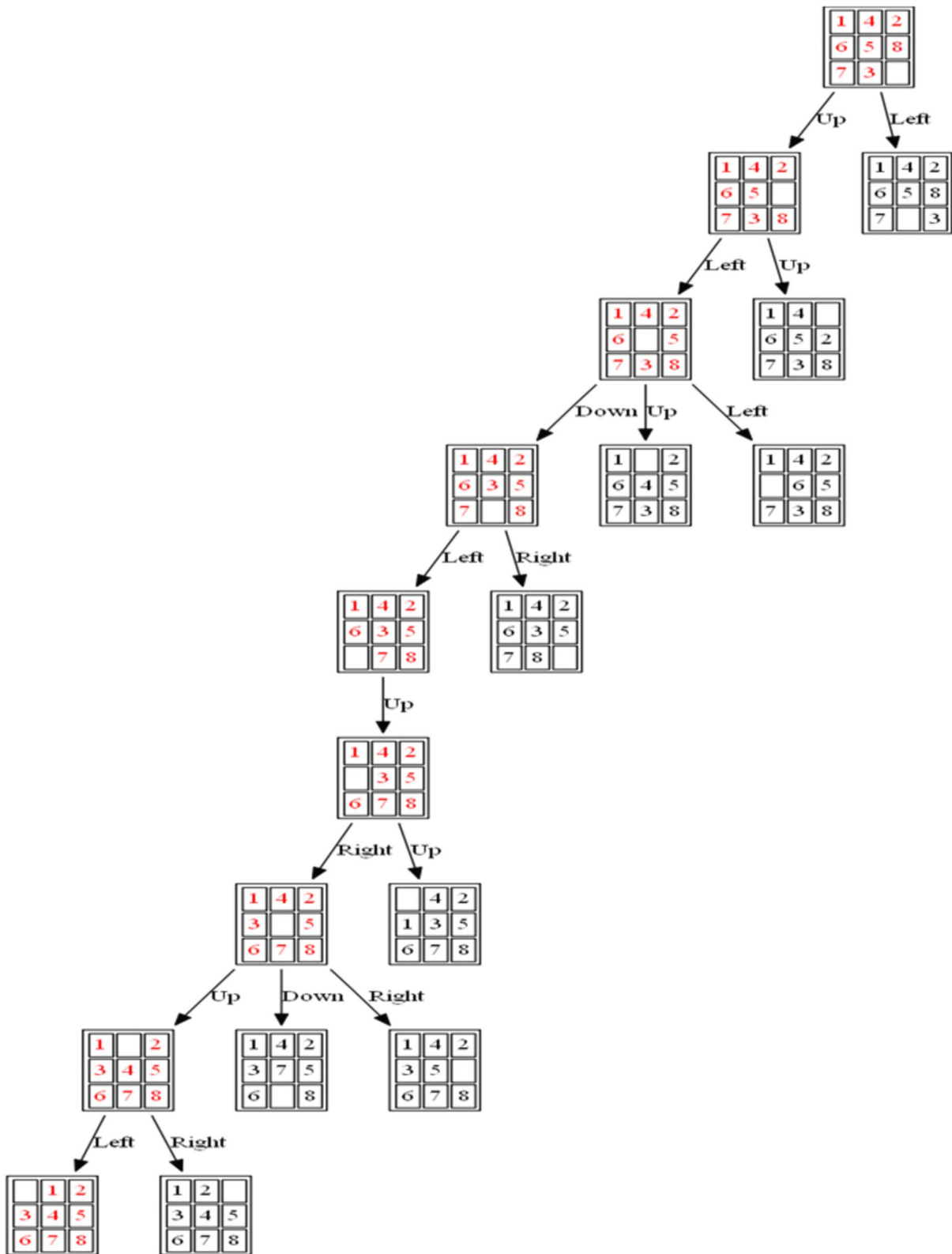


Fig. 9 A* search tree for second problem instance

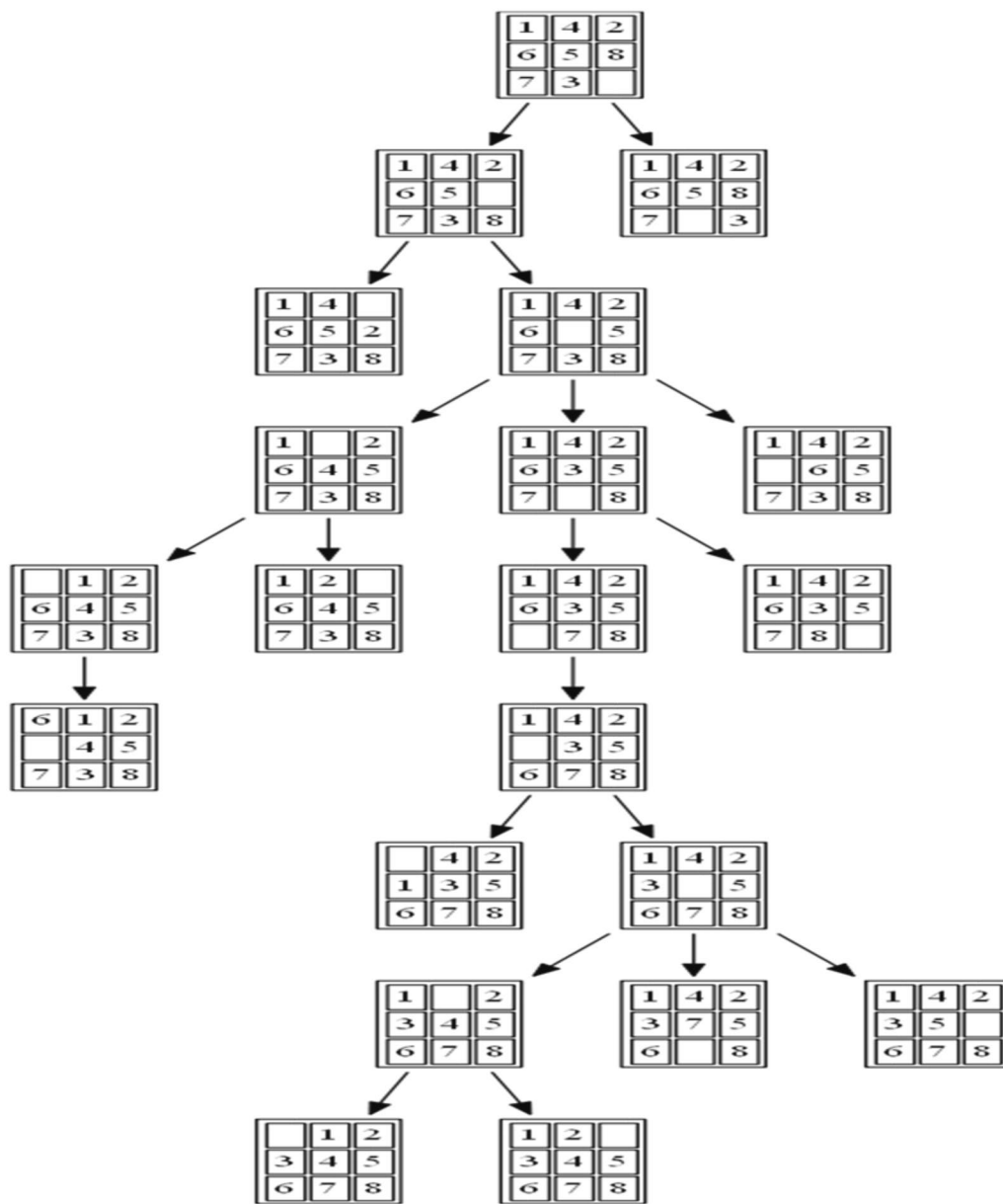


Fig. 10 A* full expanded nodes search tree

8-puzzle issue. As long as the heuristic function is admissible and consistent, the execution time should be reasonable for the majority of random initial states and goal states. The execution time, however, may be unreasonably long in certain pathological situations, such as severely restricted search spaces or badly constructed heuristic functions where the branching factor, or the total number of potential moves, is expressed as B/b (B or b). The goal state's depth is represented by D/d (D or d); while, the maximum depth of the search tree is represented by M/m (M or m). The metrics utilized in

the table are briefly explained as follows: Time Complexity: The length of time needed to solve a problem. Space Complexity: The amount of memory needed to solve a problem. Completeness: The degree to which the method is certain to locate a solution, should one exist. The degree to which an algorithm is guaranteed to find the best course of action or the shortest path to the desired state is known as its optimality.

The degree to which the heuristic function of the algorithm is accepted (i.e., never overestimating the true cost to reach the objective state) is known as admissibility.

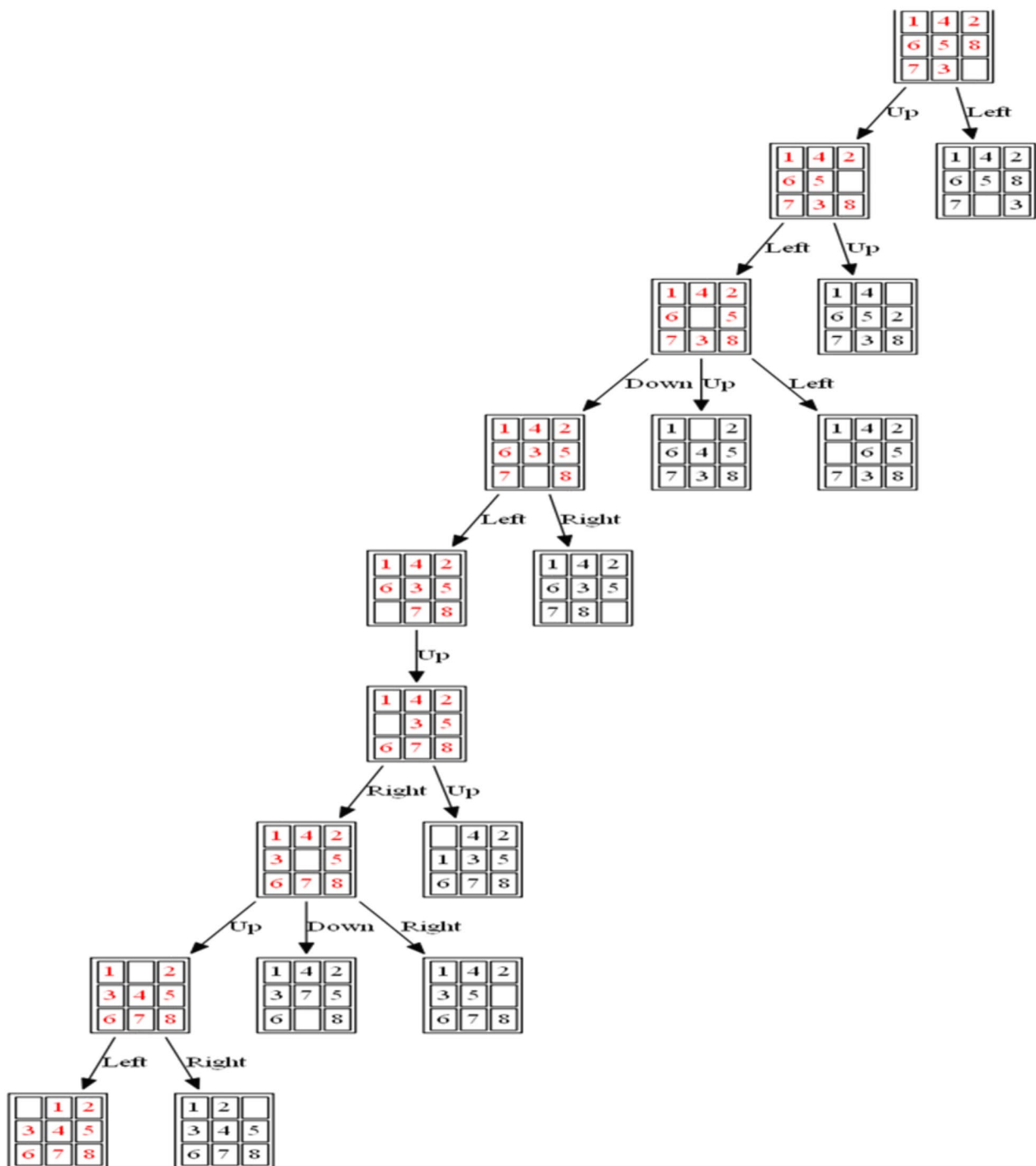
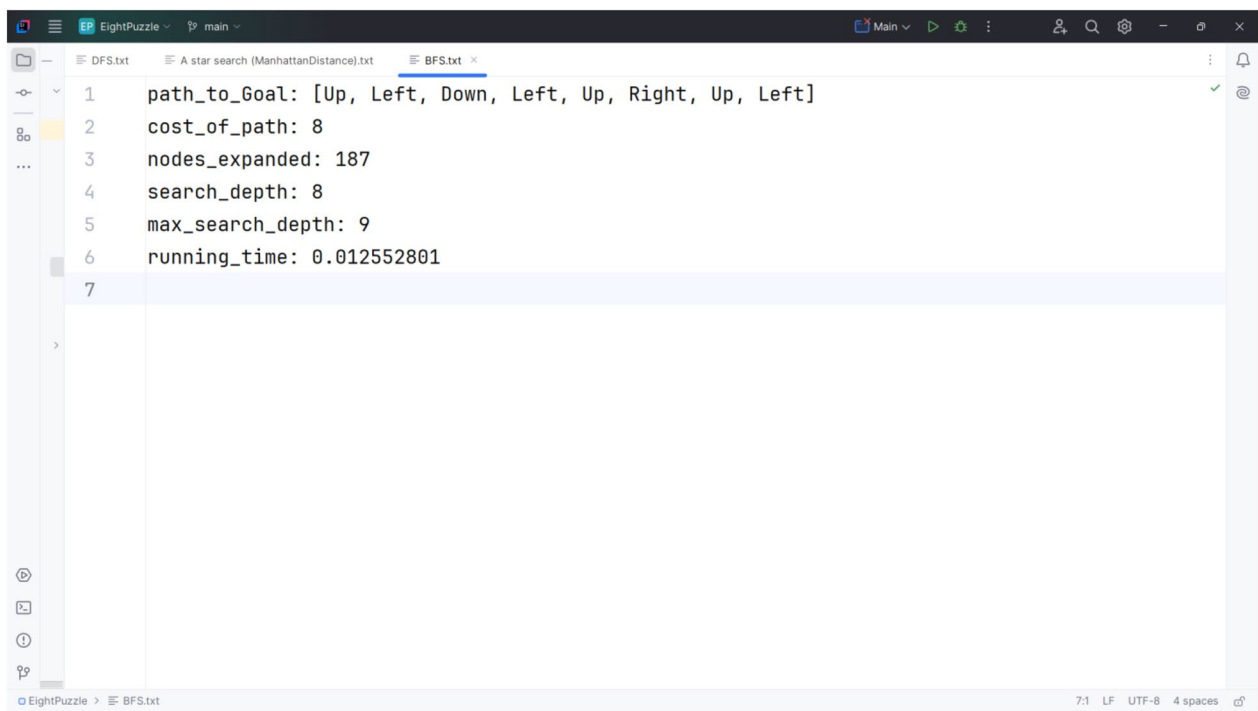


Fig. 11 BFS search tree for second problem instance

All things considered, the most effective method for resolving the 8-puzzle problem in terms of admissibility, completeness, optimality, and time and space complexity is still A* search combined with the Manhattan Distance heuristic.

Conclusions

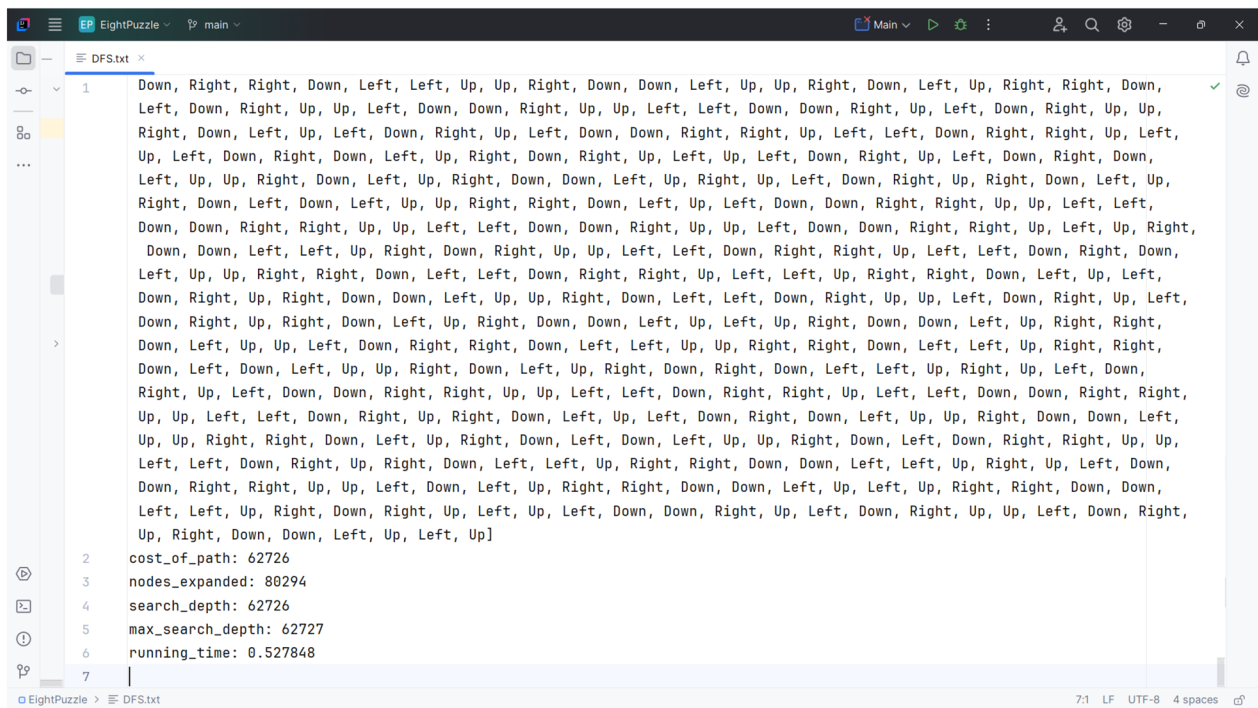
In this study, three AI search algorithm (Breadth First, Depth First and A*) were deployed in solving the 8 puzzle problem so as to evaluate their performances based on the following criteria: *optimality, completeness and*



The screenshot shows a code editor with three tabs: 'DFS.txt', 'A star search (ManhattanDistance).txt', and 'BFS.txt'. The 'BFS.txt' tab is active, displaying the following text:

```
1 path_to_Goal: [Up, Left, Down, Left, Up, Right, Up, Left]
2 cost_of_path: 8
3 nodes_expanded: 187
4 search_depth: 8
5 max_search_depth: 9
6 running_time: 0.012552801
7
```

The status bar at the bottom indicates 'EightPuzzle > BFS.txt', '7:1 LF UTF-8 4 spaces', and a file icon.

Fig. 12 BFS Search implementation on test case result

The screenshot shows a code editor with two tabs: 'DFS.txt' and 'BFS.txt'. The 'DFS.txt' tab is active, displaying the following text:

```
1 Down, Right, Right, Down, Left, Left, Up, Up, Right, Down, Down, Left, Up, Up, Right, Down, Left, Up, Right, Right, Down,
  Left, Down, Right, Up, Up, Left, Down, Down, Right, Up, Up, Left, Left, Down, Down, Right, Up, Left, Down, Right, Up, Up,
  Right, Down, Left, Up, Left, Down, Right, Up, Left, Left, Down, Right, Right, Up, Left,
  Up, Left, Down, Right, Down, Left, Up, Right, Down, Right, Up, Left, Up, Left, Down, Right, Up, Left, Down, Right, Down,
  Left, Up, Up, Right, Down, Left, Up, Right, Down, Down, Left, Up, Right, Up, Left, Down, Right, Up, Right, Down, Left, Up,
  Right, Down, Left, Down, Left, Up, Up, Right, Right, Down, Left, Up, Left, Down, Down, Right, Right, Up, Up, Left, Left,
  Down, Down, Right, Right, Up, Up, Left, Left, Down, Down, Right, Up, Up, Left, Down, Down, Right, Right, Up, Left, Up, Right,
  Down, Down, Left, Left, Up, Right, Down, Right, Up, Up, Left, Left, Down, Right, Right, Up, Left, Left, Down, Right, Down,
  Left, Up, Up, Right, Right, Down, Left, Left, Down, Right, Right, Up, Left, Left, Up, Right, Right, Down, Left, Up, Left,
  Down, Right, Up, Right, Down, Down, Left, Up, Up, Right, Down, Left, Left, Down, Right, Up, Up, Left, Down, Right, Up, Left,
  Down, Right, Up, Right, Down, Down, Left, Up, Left, Up, Right, Down, Down, Left, Up, Right, Right,
  Down, Left, Up, Up, Left, Down, Right, Right, Down, Left, Left, Up, Up, Right, Right, Down, Left, Left, Up, Right, Right,
  Down, Left, Down, Left, Up, Up, Right, Down, Left, Up, Right, Down, Right, Down, Left, Left, Up, Right, Up, Left, Down,
  Right, Up, Left, Down, Right, Right, Up, Up, Left, Down, Left, Up, Right, Right, Down, Down, Left, Up, Left, Up, Right, Right, Down, Down,
  Left, Left, Up, Right, Down, Right, Up, Left, Up, Left, Down, Down, Right, Up, Left, Down, Right, Up, Up, Left, Down, Right,
  Up, Right, Down, Down, Left, Up, Left, Up]
```

```
2 cost_of_path: 62726
3 nodes_expanded: 80294
4 search_depth: 62726
5 max_search_depth: 62727
6 running_time: 0.527848
7
```

The status bar at the bottom indicates 'EightPuzzle > DFS.txt', '7:1 LF UTF-8 4 spaces', and a file icon.

Fig. 13 DFS Search implementation on test case result

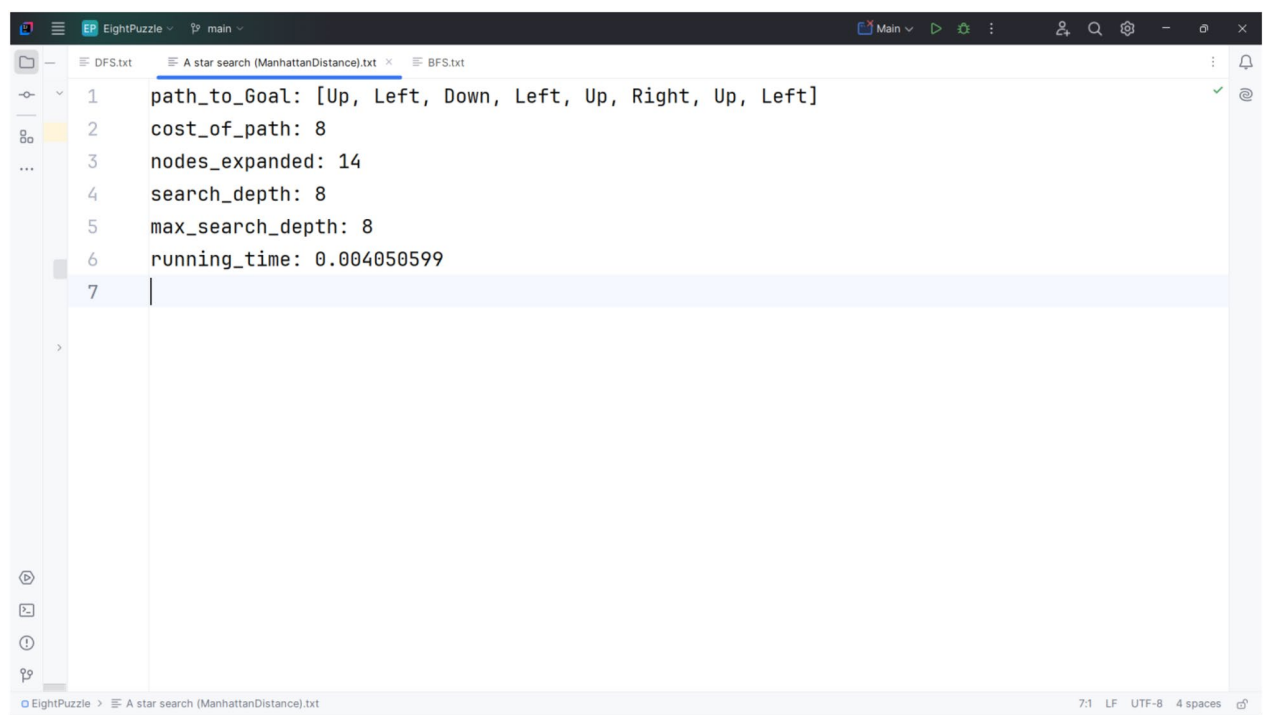


Fig. 14 A* implementation result using Manhattan on test case

efficiency (time and space). From the test data, it can be concluded that the A* algorithm is the best of the three when solving the 8 puzzle problem. Although in comparison with other methods, such as Greedy Best First Search, it may not necessarily be the fastest for other puzzles. In terms of what the puzzle looks like, how the tiles move, what the goal state should be and how to keep track of the puzzle states, A* is the best search method. It will find the optimal solution and do so in a reasonable amount of time with lesser memory, especially using the Manhattan Distance. BFS will always find a result, and often quite quickly for smaller puzzles. But, if the goal is deep into the tree and close to the state, DFS should be used. Within the application, additional option is provided to users, to try out different configurations of puzzles and even attempt to expand on the 8 Puzzle in order to create a 15 Puzzle solver and implement the search algorithms to the test.

Recommendation

While significant progress has been made in solving this 8 problem, there are still several challenges and limitations in this area of research. There is a need to develop algorithms that can handle non-deterministic environments as well as design algorithms that can handle multi-agent problems. There is also a need to

extend the 8-puzzle problem to larger board sizes and other problem domains, thereby, exploring the use of machine learning techniques in improving the general performance of search algorithms in solving this problem.

Abbreviations

AI	Artificial intelligence
DFS	Depth-first search
BFS	Breadth-first search
UCS	Uniform cost search
IDA	Iterative-deepening A*
NP-hard	Non-deterministic polynomial-time hard
A	A search
$h(n)$	Heuristic function
$f(n)$	Cost function
$c(n)$	Actual cost

Acknowledgements

Not applicable.

Author contributions

BGB, IBD and OJP prepared the manuscript and approved the submitted version; BA and ODT contributed substantially to the research design and review process; while, MA participated in data collection and analysis. All authors have read and approved the final manuscript.

Funding

No funding received.

Availability of data and materials

Not applicable.

Declarations

Ethics approval and consent to participate

This study adhered to the National Research Policy and Operational Guidelines. Thus, the researchers involved in designing the study, collecting and analyzing the data and producing this manuscript followed all ethical research conducts.

Consent for publication

Not applicable.

Competing interests

The authors declare that they have no competing interests.

Received: 16 August 2024 Accepted: 26 October 2024

Published online: 18 November 2024

References

- Awerbuch B (1985) Complexity of network synchronization. *J ACM* 32(4):804–823
- Chowdhary KR, Chowdhary KR (2020) Introducing artificial intelligence. *Fundam Artif Intell* 1–23
- Gulak AM (2024) Research paper by Gulak 23 September 2024 Artificial Intelligence. *Artif Intell*
- Jordan AE (2016) A comparative study of three heuristic functions used to solve the 8-puzzle. *Br J Math Comput Sci* 16(1):1–18. <https://doi.org/10.9734/bjmcs/2016/24467>
- Jordan AE (2018) A comparative study of the A* heuristic search algorithm used to solve efficiently a puzzle game. In: *IOP conference series: materials science and engineering*, vol 294. IOP Publishing
- Nayak D (2014) Analysis and implementation of admissible heuristics in 8 puzzle problem (Doctoral dissertation). <http://ethesis.nitrkl.ac.in/5575/>. Accessed 12 Aug 2023
- Meister C (2020) Best-first beam search. *Trans Assoc Comput Linguist* 8:795–809
- Menon V, Amali GB (2018) Performance analysis of various uninformed and informed search strategies on 8 puzzle problems—a case study. *World Wide J Multidiscip Res Dev* 4(12):96–99
- Mishra AK, Siddalingaswamy PC (2017) Analysis of tree based search techniques for solving 8-puzzle problem. In: *2017 Innovations in power and advanced computing technologies (i-PACT)*
- Navya P, Ranjith R (2021) Performance analysis of BFS and DFS algorithms for food serving robot in an eatery. *EasyChair*
- Norvig P, Russell SJ (2010) *Intelligenza artificiale. Un approccio moderno*
- Nosrati K, Van Den Eeckhaut M (2012) Assessment of groundwater quality using multivariate statistical techniques in Hashtgerd Plain, Iran. *Environ Earth Sci* 65:331–344
- Omonkhodion G (2023) A comparative study of A* and greedy best-first search algorithms in solving 8-puzzle game. *Int J Soc Sci Sci Stud* 3(1):2321–2329
- Pathak ML, Patel RL, Rami SP (2018) Comparative analysis of search algorithms. *Int J Comput Appl* 179(50):40–43. <https://doi.org/10.5120/ijca2018917358>
- Shi H (2011) Searching algorithms implementation and comparison of Eight-puzzle problem. In: *Proceedings of 2011 International conference on computer science and network technology*, vol 2. IEEE, pp 1203–1206
- Tarjan R (1972) Depth-first search and linear graph algorithms. *SIAM J Comput* 1(2):146–160. <https://doi.org/10.1137/0201010>
- Zhang W, Sauppe JJ, Jacobson SH (2021) Comparison of the number of nodes explored by cyclic best first search with depth contour and best first search. *Comput Oper Res* 126:105–129. <https://doi.org/10.1016/j.cor.2020.105129>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.